TL03 Regresión logística

Índice

1. Introducción
2. Regresión logística
3. Ingeniería de características
4. Aplicación a MNIST
5. Ejercicio

1 Introducción

Clasificador discriminativo: modela **posteriors** directamente; no requiere priors ni densidades condicionales

$$p(y = c \mid \boldsymbol{x}; \boldsymbol{\theta}) = \cdots$$

Ingeniería de características: la entrada $\boldsymbol{x}$ se sustituye por alguna transformación $\boldsymbol{\phi}(\boldsymbol{x})$

Objetivo: aplicar regresión logística con ingeniería de características

2 Regresión logística

Regresión logística con ingeniería de características: $\mathbf{W} \in \mathbb{R}^{D \times C}$ es la matriz de pesos

$$p(y \mid \boldsymbol{x}, \mathbf{W}) = \mathrm{Cat}(y \mid \boldsymbol{\mu}) \qquad \text{con} \qquad \boldsymbol{\mu} = \mathrm{Cat}(y \mid \mathcal{S}(\boldsymbol{a})) \qquad \text{y} \qquad \boldsymbol{a} = \mathbf{W}^t \boldsymbol{x}$$

donde $\mathbf{W}$ se ajusta minimizando la NLL mediante descenso por gradiente

Regresión logística en scikit-learn: `LogisticRegression`

- `tol=1e-4`: tolerancia del criterio de parada
- `C=1.0`: paramétro de des-regularización (cuanto mayor, menor regularización)
- `fit_intercept=True`: añade un parámetro de sesgo a $\boldsymbol{a}$
- `solver='lbfgs'`: técnica de optimización (descenso por gradiente)
- `max_iter=100`: máximo número de iteraciones

3 Ingeniería de características

Objetivo: encontrar alguna transformación de los datos, $\phi(\boldsymbol{x})$, que linearice un problema de clasificación con clases (datos) no linealmente separables

Características polinómicas: $\boldsymbol{x}$ se sustituye por una expansión polinómica hasta un grado dado

Características polinómicas en scikit-learn: `PolynomialFeatures`

- `degree=2`: grado máximo de las características generadas, o par "(mínimo, máximo)"
- `interaction_only=False`: características interactivas (productos de características distintas) solo
- `include_bias=True`: notación homogénea

Ejemplo: $\boldsymbol{x} = (x_1, x_2)^t \Rightarrow (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)^t$, $(1, x_1, x_2, x_1x_2)^t$ y $(1, x_1, x_2, x_1^2, x_2^2)^t$

In [ ]:
```
import numpy as np; from sklearn.preprocessing import PolynomialFeatures
X = np.arange(2, 4).astype(np.float64).reshape(-1, 2)
print(X, '=>', PolynomialFeatures(2).fit_transform(X))
```

```
[[2. 3.]] => [[1. 2. 3. 4. 6. 9.]]
```

In [ ]:
```
print(X, '=>', PolynomialFeatures(2, interaction_only=True).fit_transform(X))
```

```
[[2. 3.]] => [[1. 2. 3. 6.]]
```

In [ ]:
```
print(X, '=>', np.c_[(np.ones((X.shape[0], 1)), X, np.square(X))])
```

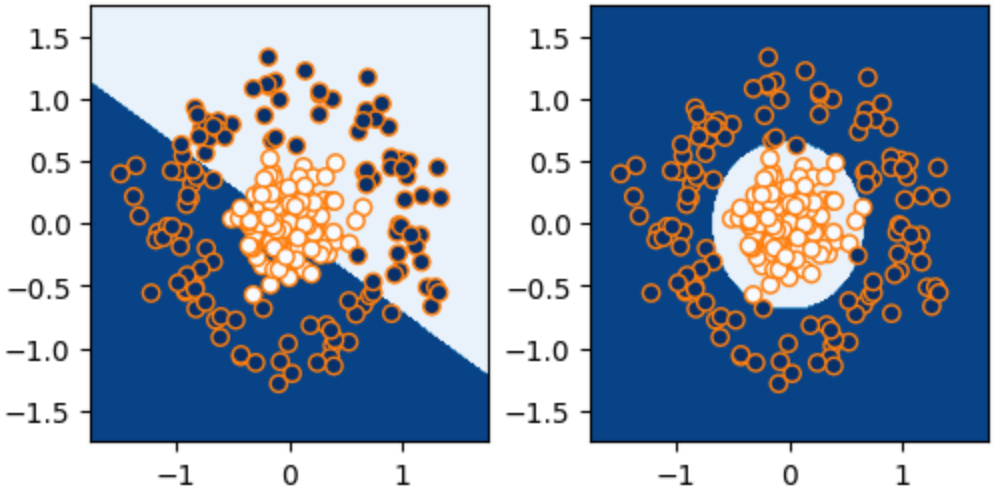
```
[[2. 3.]] => [[1. 2. 3. 4. 9.]]
```

Ejemplo: tarea sintética con clases no linealmente separables, sin y con ingeniería de características

In [ ]:
```
from sklearn.model_selection import train_test_split; from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, PolynomialFeatures; from sklearn.pipeline import make_pipeline
from sklearn.datasets import make_circles
X, y = make_circles(n_samples=1000, factor=0.2, noise=0.2, random_state=23)
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=23)
clf = LogisticRegression(random_state=23).fit(X_train, y_train); acc = clf.score(X_test, y_test)
clfp = make_pipeline(StandardScaler(), PolynomialFeatures(), LogisticRegression(random_state=23))
clfp.fit(X_train, y_train); accp = clfp.score(X_test, y_test)
print(f'Precisión: {acc:.1%}, con características polinómicas {accp:.1%}');
```

```
Precisión: 50.4%, con características polinómicas 99.2%
```

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
R = np.linspace(-1.75, 1.75, 200); X1, X2 = np.meshgrid(R, R); X = np.c_[np.ravel(X1), np.ravel(X2)]
fig, (ax, axp) = plt.subplots(1, 2, figsize=(5, 2.5), constrained_layout=True)
ax.contourf(X1, X2, clf.predict(X).reshape(X1.shape), 6, cmap='Blues_r')
ax.scatter(*X_test.T, c=y_test, cmap='Blues_r', edgecolors='C1')
axp.contourf(X1, X2, clfp.predict(X).reshape(X1.shape), 6, cmap='Blues_r')
axp.scatter(*X_test.T, c=y_test, cmap='Blues_r', edgecolors='C1');
```



4 Aplicación a MNIST

Memoria: escogemos $K = 300$ ya que las características cuadráticas consumen mucha memoria

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("ylecun/mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
K = 300; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train)
```

In [ ]:
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
import time; start = time.time()
poly = PolynomialFeatures(2); X_train = poly.fit_transform(X_train)
print(X_train.shape, y_train.shape)
clf = LogisticRegression(C=0.1, max_iter=200).fit(X_train, y_train)
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
del X_train, y_train
```

```
(60000, 45451) (60000,)
Tiempo (hh:mm:ss): 00:01:35
```

In [ ]:
```
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
X_test = pca.transform(X_test); X_test = poly.transform(X_test)
print(X_test.shape, y_test.shape)
acc = clf.score(X_test, y_test)
print(f'Precisión: {acc:.2%}')
```

```
(10000, 45451) (10000,)
Precisión: 98.32%
```

Mejora: el 98.3% es claramente mejor que el 96% de QDA

5 Ejercicio

Ejercicio: aplica ingeniería de características a Fashion-MNIST y CIFAR-10 siguiendo el ejemplo dado para MNIST

Aplicación a Fashion-MNIST: con $K = 300$

- Precisión en test: 89.5%
- Mejor que el 84.5% de regresión logística sin ingeniería de características

Aplicación a CIFAR-10: con $K = 300$

- Precisión en test: algo más del 52%
- Mejor que el 41% de regresión logística sin ingeniería de características; similar a QDA

Fashion-MNIST

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("zalando-datasets/fashion_mnist").with_format("numpy")
X_train = ds['train'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
K = 300; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train)
```

In [ ]:
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
import time; start = time.time()
poly = PolynomialFeatures(2); X_train = poly.fit_transform(X_train)
print(X_train.shape, y_train.shape)
clf = LogisticRegression(C=0.1, max_iter=200).fit(X_train, y_train)
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
del X_train, y_train
```

```
(60000, 45451) (60000,)
Tiempo (hh:mm:ss): 00:07:35
```

In [ ]:
```
X_test = ds['test'][:]['image'].astype(np.float32).reshape(-1, 28*28) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
X_test = pca.transform(X_test); X_test = poly.transform(X_test)
print(X_test.shape, y_test.shape)
acc = clf.score(X_test, y_test)
print(f'Precisión: {acc:.2%}')
```

```
(10000, 45451) (10000,)
Precisión: 89.54%
```

CIFAR-10

In [ ]:
```
import numpy as np; import datasets; from sklearn.decomposition import PCA
ds = datasets.load_dataset("uoft-cs/cifar10").with_format("numpy")
X_train = ds['train'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_train = ds['train'][:]['label'].astype(np.uint8)
K = 300; pca = PCA(n_components=K); X_train = pca.fit_transform(X_train)
```

In [ ]:
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
import time; start = time.time()
poly = PolynomialFeatures(2); X_train = poly.fit_transform(X_train)
print(X_train.shape, y_train.shape)
clf = LogisticRegression(C=0.1, max_iter=200).fit(X_train, y_train)
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
del X_train, y_train
```

```
(50000, 45451) (50000,)
Tiempo (hh:mm:ss): 00:05:46
```

In [ ]:
```
X_test = ds['test'][:]['img'].astype(np.float32).reshape(-1, 32*32*3) / 255.
y_test = ds['test'][:]['label'].astype(np.uint8)
X_test = pca.transform(X_test); X_test = poly.transform(X_test)
print(X_test.shape, y_test.shape)
acc = clf.score(X_test, y_test)
print(f'Precisión: {acc:.2%}')
```

```
(10000, 45451) (10000,)
Precisión: 52.30%
```